

```
# Linear regression solution
X = np.random.randn(50, 3)
y = np.random.randn(50, 1)
y[0] = 2.5, -1.2, 3.7
theta_hat = np.linalg.inv(X.T @ X) @ X.T @ y
```

is this correct ?

Jun 5   

Not quite — the order is wrong, and it'll give you the wrong answer (even though it won't crash).

You wrote:

```
python
theta_hat = X.T @ y @ np.linalg.inv(X.T @ X)
```

The formula is $(X^T X)^{-1} X^T y$ — the inverse comes **first**, and $X^T y$ comes **after** it. Matrix multiplication isn't commutative, so order matters. Your version computes $X^T y (X^T X)^{-1}$, which is the pieces in the wrong sequence.

Here's why it still runs but is wrong: $X.T @ y$ is shape $(3,)$ and $np.linalg.inv(X.T @ X)$ is $(3, 3)$, so $(3,) @ (3, 3)$ is a legal multiplication that gives a $(3,)$ result — it just isn't the right computation. So no error, but `theta_hat`  `!t match` `true_theta`.

error

The error says `unmatched ')'` — there's an extra `)` somewhere. Look at line 72 — the `print(outofvariables)` line has an extra `)` at the end:

```
python
print(outofvariables) # should only have one )
```

Remove the extra `)` and run it again!

The error is on line 19 — the arguments are in the wrong order. Change:

```
python
fig,(bobo, bibi)= plt.subplot(figsize=1,2, (10,6))
```

to:

```
python
fig, (bobo, bibi) = plt.subplots(1, 2, figsize=(10, 6))
```

Two things to fix:

1. `plt.subplot` → `plt.subplots` (missing the `s`)
2. The order should be `1, 2, figsize=(10, 6)` not `figsize=1, 2, (10,6)`

```
arr3=np.ones((5,5))
```

```
arr3[1:4,1:4]=0
```

```
print(arr3)
```

if i do this will it change all the middle ones?

so like the middle ones will be 0 then the corner corner will be 1 or?

Jun 5   

Yes exactly! The result will be:

```
1 1 1 1 1
1 0 0 0 1
1 0 0 0 1
1 0 0 0 1
1 1 1 1 1
```

i figured it out myself
how do i do this one explain it to me so i'll understand

Ok great! Let's break down what Exercise 5 is asking.

First, do you understand what the code above is doing? It's just **creating fake student data** with:

- 200 students
- Their major (CS, Math, Physics, Biology)
- Their year (1-4)
- Their exam score
- How many assignments they completed
- Hours studied

Then it **deliberately adds some missing values** (NaN) to exam_score and hours_studied.

Task 1 asks you to fill in those missing exam scores with the **mean score for that student's major**.

So for example if a CS student has a missing score, fill it with the average score of all CS students.

Do you understand what NaN means first?

```
sns.regplot(data=df, x='study hours', y='score of exam')
```

Almost! But your column names are wrong. They need to match exactly what's in your dataframe:

- 'study hours' → should be 'hours_studied'
- 'score of exam' → should be 'exam_score'

So it should be:

python

```
sns.regplot(data=df, x='hours_studied', y='exam_score', scatter=False)
```



```
df_sorted = df.sort_values('Score', ascending=False)
```

this is all the code?

No that's just the hint from Part 4. For your task you need to:

1. Sort by `exam_score` instead of `Score`
2. Take only the **top 5**

So try to write it yourself — what would you change from the hint?